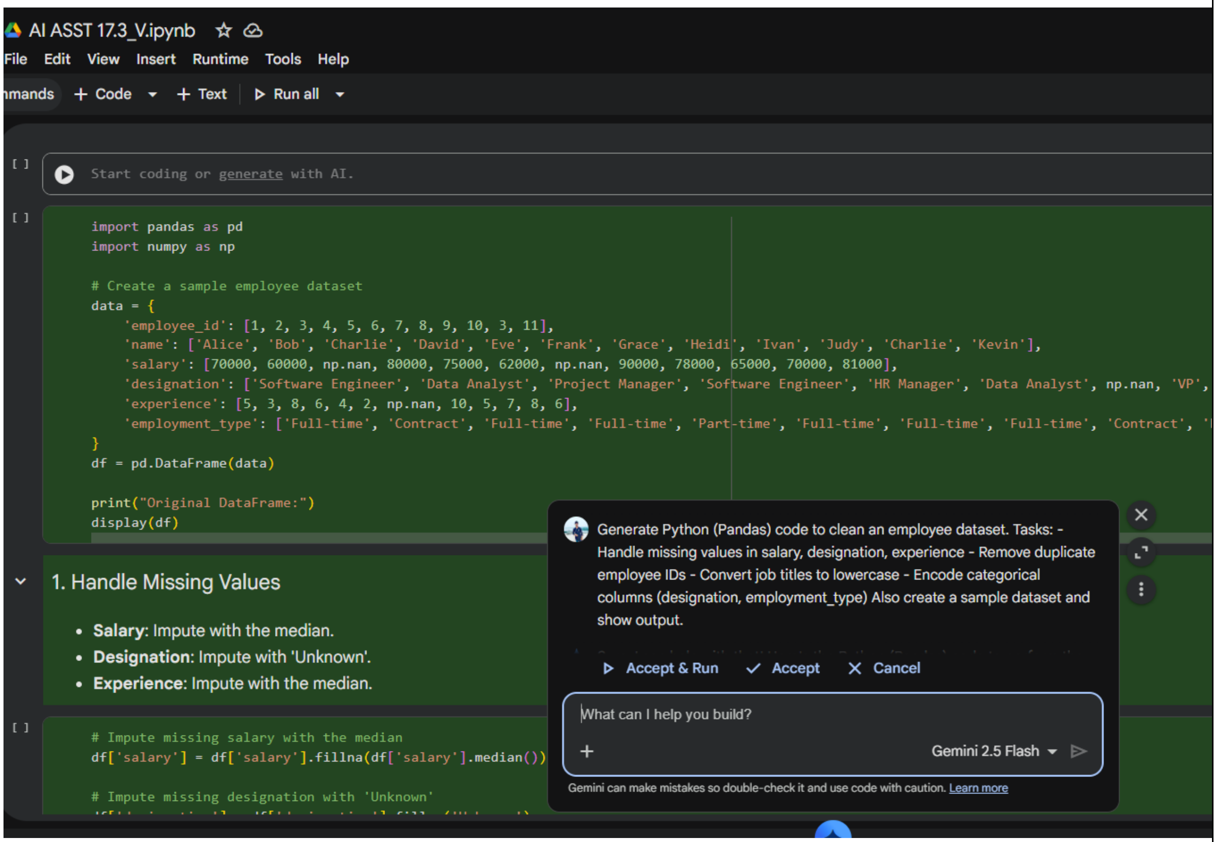
Harshavardhan
2303A51744

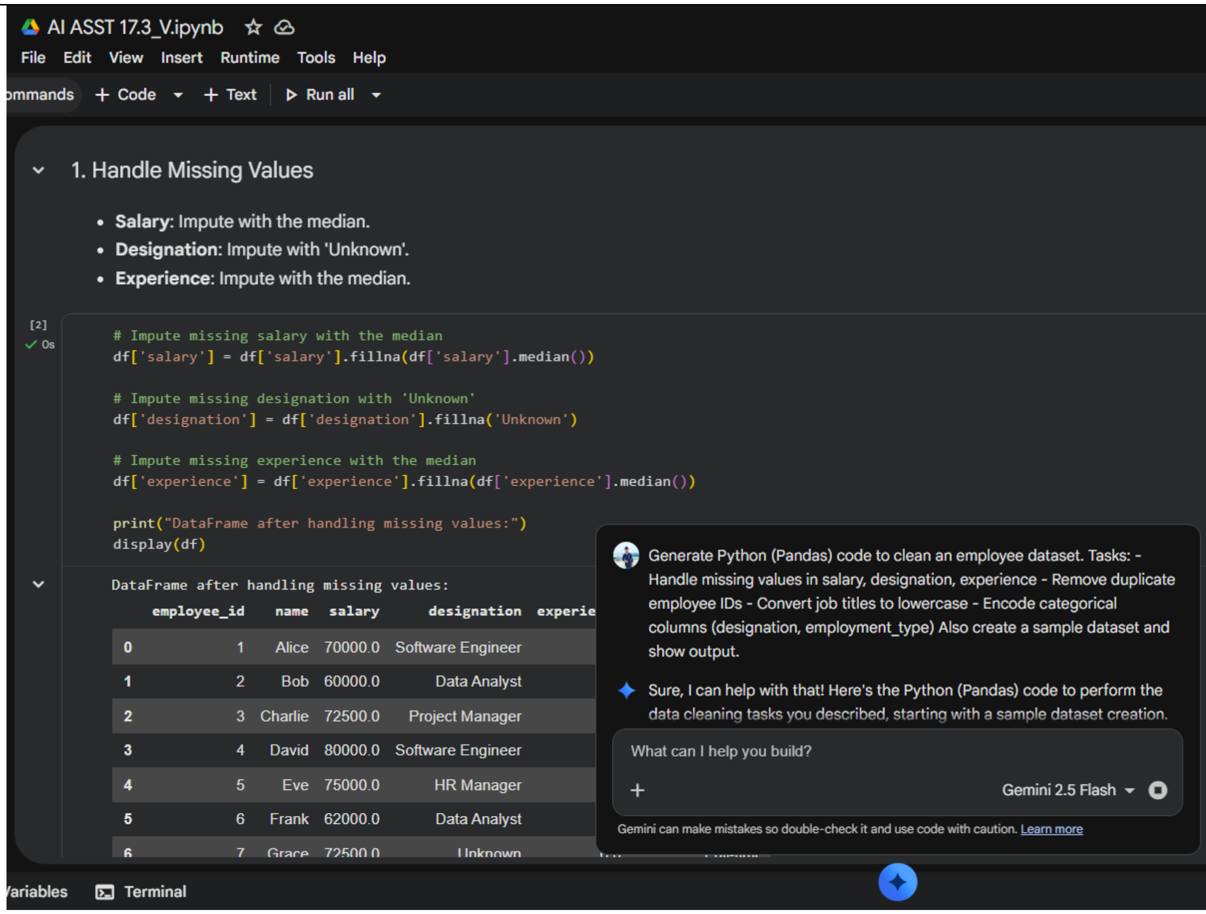
B25

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE			DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**		**Academic Year:**2025-202
Course Coordinator Name		Dr. Rishabh Mittal		
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding	
Year/Sem	III/II	**Regulation**	R23	
Date and Day of Assignment	**Week9 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52	
Duration	2 Hours	**Applicable to Batches**	All batches	
Assignment Number:17.3(Present assignment number)/**24**(Total number of assignments)				

	Question	*Expected Tim*

		e to complete
1	**Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts** **Lab Objectives:** • Learn how to clean raw datasets using AI-assisted Python scripting. • Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization. • Automate repetitive data-cleaning tasks with AI-generated code. • Understand how preprocessing impacts model performance. **Task 1 Employee Dataset Cleaning** **Task:** Use AI assistance to clean an employee information dataset. **Instructions:** • Handle missing values in salary, designation, and experience • Remove duplicate employee IDs • Standardize text values (job titles in lowercase) • Encode categorical features (designation, employment_type) **Expected Output:** A clean dataset suitable for HR analytics and prediction tasks. Prompt Generate Python (Pandas) code to clean an employee dataset. Tasks: - Handle missing values in salary, designation, experience - Remove duplicate employee IDs - Convert job titles to lowercase - Encode categorical columns (designation, employment_type) Also create a sample dataset and show output.	Week9 - Wednesday


AI ASST 17.3_V.ipynb
File Edit View Insert Runtime Tools Help
+ Code + Text Run all
Start coding or generate with AI.
import pandas as pd
import numpy as np
# Create a sample employee dataset
data = {
'employee_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 3, 11],
'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy', 'Charlie', 'Kevin'],
'salary': [70000, 60000, np.nan, 80000, 75000, 62000, np.nan, 90000, 78000, 65000, 70000, 81000],
'designation': ['Software Engineer', 'Data Analyst', 'Project Manager', 'Software Engineer', 'HR Manager', 'Data Analyst', np.nan, 'VP',
'experience': [5, 3, 8, 6, 4, 2, np.nan, 10, 5, 7, 8, 6],
'employment_type': ['Full-time', 'Contract', 'Full-time', 'Full-time', 'Part-time', 'Full-time', 'Full-time', 'Full-time', 'Contract',
}
df = pd.DataFrame(data)
print("Original DataFrame:")
display(df)
1. Handle Missing Values
Salary: Impute with the median.
Designation: Impute with 'Unknown'.
Experience: Impute with the median.
Impute missing salary with the median
df['salary'] = df['salary'].fillna(df['salary'].median())
Impute missing designation with 'Unknown'
Generate Python (Pandas) code to clean an employee dataset. Tasks: - Handle missing values in salary, designation, experience - Remove duplicate employee IDs - Convert job titles to lowercase - Encode categorical columns (designation, employment_type) Also create a sample dataset and show output.
Accept & Run
Accept
Cancel
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes so double-check it and use code with caution. Learn more



Explanation

- Missing salary → replaced with **mean**
- Experience → replaced with **median**
- Duplicate IDs → removed using `drop_duplicates`
- Text standardized using `.str.lower()`
- Categorical encoding → `get_dummies()`

Task 2 – Student Performance Data Preprocessing
Task:
Preprocess student performance data using AI assistance.
Instructions:

- Convert exam_date column into datetime format
- Extract new features (month, semester, day)
- Normalize total_marks
- Detect and handle outliers in marks using IQR or Z-score

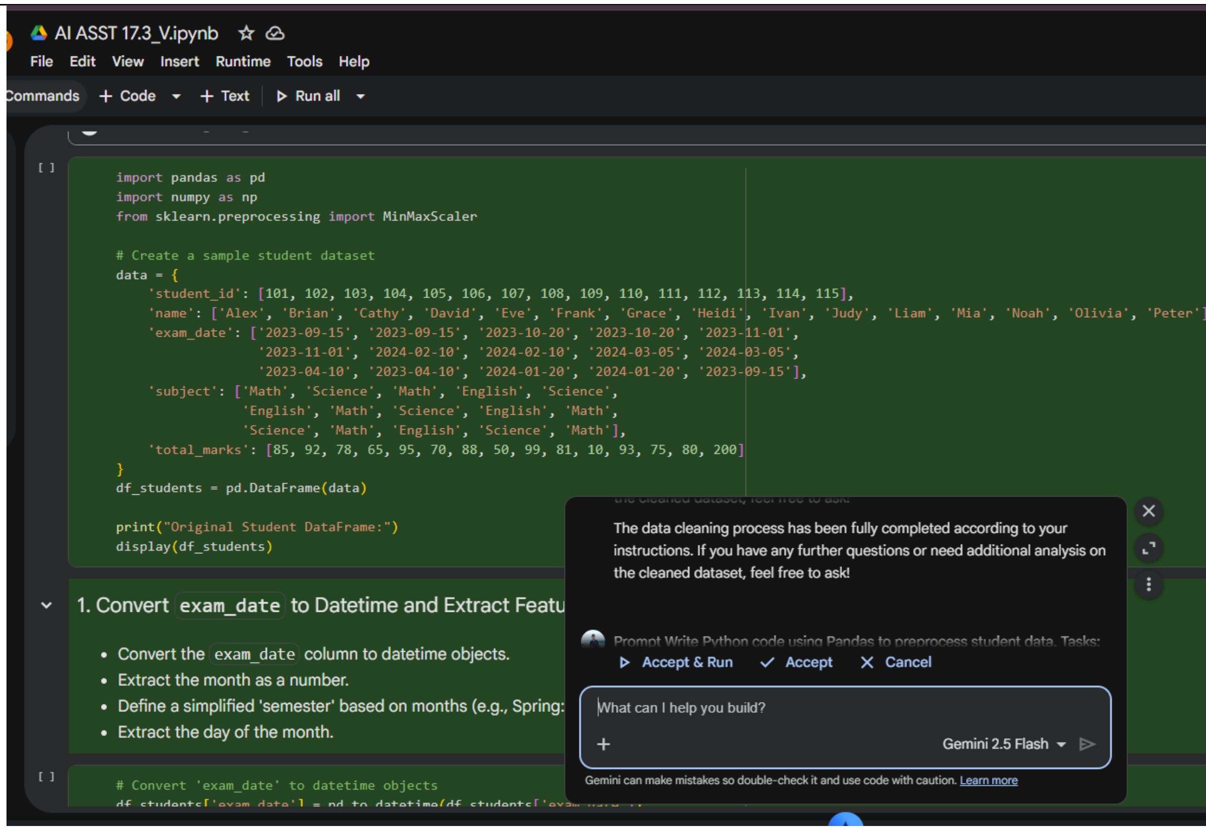
Expected Output:
A transformed dataset with time-based academic features and normalized marks.

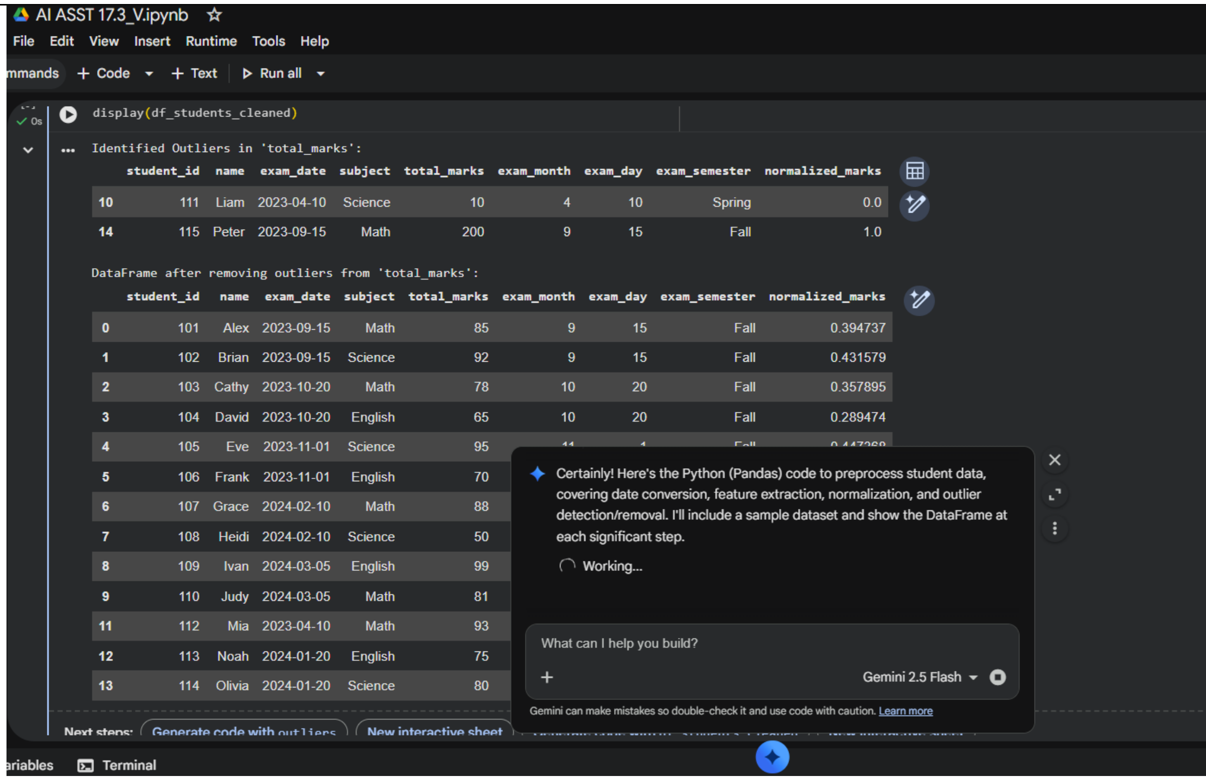
Prompt

```
Write Python code using Pandas to preprocess student data.

Tasks:
- Convert exam_date to datetime
- Extract month, semester, day
- Normalize total_marks
- Detect outliers using IQR and remove them

Include sample data.
```


AI ASST 17.3_V.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
# Create a sample student dataset
data = {
'student_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115],
'name': ['Alex', 'Brian', 'Cathy', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy', 'Liam', 'Mia', 'Noah', 'Olivia', 'Peter'],
'exam_date': ['2023-09-15', '2023-09-15', '2023-10-20', '2023-10-20', '2023-11-01',
'2023-11-01', '2024-02-10', '2024-02-10', '2024-03-05', '2024-03-05',
'2023-04-10', '2023-04-10', '2024-01-20', '2024-01-20', '2023-09-15'],
'subject': ['Math', 'Science', 'Math', 'English', 'Science',
'English', 'Math', 'Science', 'English', 'Math',
'Science', 'Math', 'English', 'Science', 'Math'],
'total_marks': [85, 92, 78, 65, 95, 70, 88, 50, 99, 81, 10, 93, 75, 80, 200]
}
df_students = pd.DataFrame(data)
print("Original Student DataFrame:")
display(df_students)
1. Convert exam_date to Datetime and Extract Featu
Convert the exam_date column to datetime objects.
Extract the month as a number.
Define a simplified 'semester' based on months (e.g., Spring:
Extract the day of the month.
Convert 'exam_date' to datetime objects
The data cleaning process has been fully completed according to your instructions. If you have any further questions or need additional analysis on the cleaned dataset, feel free to ask!
Prompt Write Python code using Pandas to preprocess student data. Tasks:
Accept & Run
Accept
Cancel
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes so double-check it and use code with caution. Learn more



Explanation

- Date converted → enables feature extraction
- Semester logic → based on month
- Normalization → Min-Max scaling
- Outliers → removed using IQR method

Task 3 – Student Health Data Preparation
Task:
Clean and preprocess student health records using AI scripts.
Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

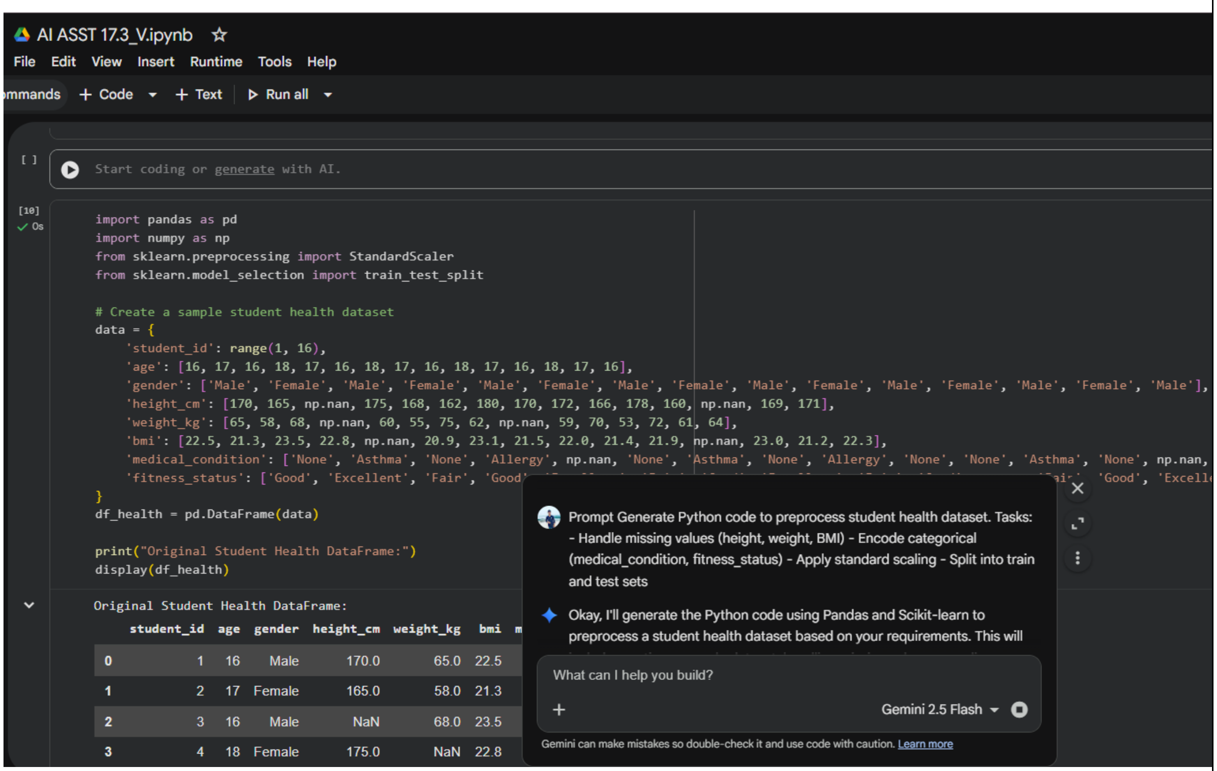
Expected Output:

A preprocessed dataset ready for predictive health analysis.

Prompt

```
Generate Python code to preprocess student health dataset.

Tasks:
- Handle missing values (height, weight, BMI)
- Encode categorical (medical_condition, fitness_status)
- Apply standard scaling
- Split into train and test sets
```



Explanation

- Missing values → replaced with mean
- Encoding → one-hot encoding
- Scaling → StandardScaler (important for ML)
- Splitting → training/testing dataset

Task 4: Real-Time Application: Ride-Sharing Data Cleaning
Scenario:
A ride-sharing company has trip data with errors.
Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

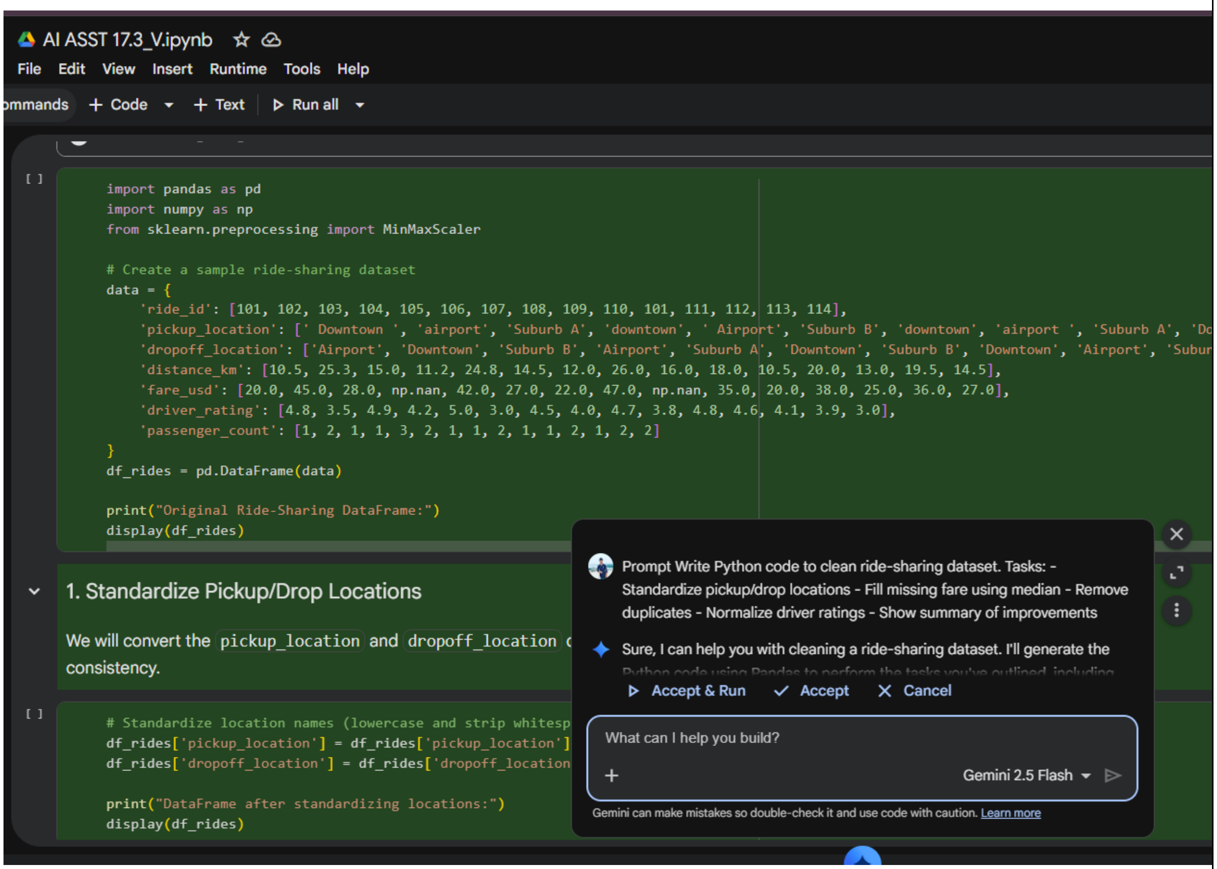
Expected Output:
A clean dataset ready for operational analysis.

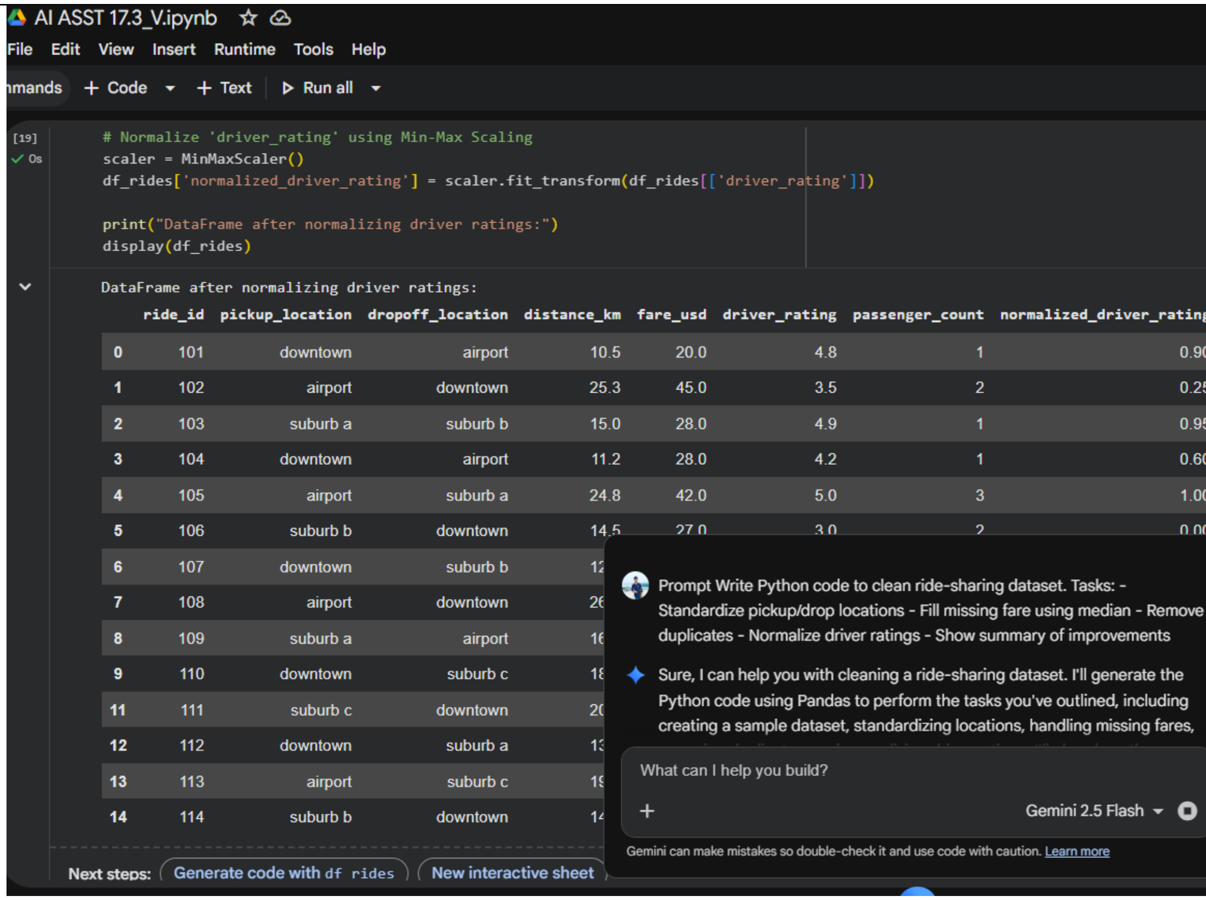
Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Prompt

Write Python code to clean ride-sharing dataset.

Tasks:
- Standardize pickup/drop locations
- Fill missing fare using median
- Remove duplicates
- Normalize driver ratings
- Show summary of improvements






```python
# Normalize 'driver_rating' using Min-Max Scaling
scaler = MinMaxScaler()
df_rides['normalized_driver_rating'] = scaler.fit_transform(df_rides[['driver_rating']])

print("DataFrame after normalizing driver ratings:")
display(df_rides)
```

DataFrame after normalizing driver ratings:

	ride_id	pickup_location	dropoff_location	distance_km	fare_usd	driver_rating	passenger_count	normalized_driver_rating
0	101	downtown	airport	10.5	20.0	4.8	1	0.90
1	102	airport	downtown	25.3	45.0	3.5	2	0.25
2	103	suburb a	suburb b	15.0	28.0	4.9	1	0.95
3	104	downtown	airport	11.2	28.0	4.2	1	0.60
4	105	airport	suburb a	24.8	42.0	5.0	3	1.00
5	106	suburb b	downtown	14.5	27.0	3.0	2	0.00
6	107	downtown	suburb b	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
7	108	airport	downtown	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
8	109	suburb a	airport	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
9	110	downtown	suburb c	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
11	111	suburb c	downtown	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
12	112	downtown	suburb a	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
13	113	airport	suburb c	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
14	114	suburb b	downtown	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

Explanation

- Location standardization → avoids inconsistency
- Missing fare → median (robust)
- Duplicate rides → removed
- Ratings → normalized for analysis